

House Price Prediction using Machine Learning Techniques



Nikolaos Belimpasakis
MEng in Computer Engineering and Informatics

Components

1. Overview and Objectives	2
2. Dataset Description	3
3. Model Implementation	5
3.1 Environment Setup	5
3.2 Importing the Dataset	7
3.3 Data Wrangling	10
3.4 Exploratory Data Analysis	12
3.5 Model Development	17
3.6 Model Evaluation and Refinement	21
4. Conclusion	25
5. References	26

1. Overview and Objectives

This project was developed within the framework of the IBM Generative AI Engineering program [1] and focuses on the application of data analysis and machine learning techniques for real-world predictive modeling. The assignment provides hands-on experience with the complete machine learning workflow, including data exploration, data preprocessing, feature engineering, model development, model evaluation, and hyperparameter optimization.

In this scenario, we assume the role of data analysts working for a Real Estate Investment Trust (REIT), a company that invests in residential properties and relies on data-driven decisions to evaluate potential investments. The organization seeks to estimate the market value of residential properties using historical housing data and various property characteristics.

The dataset used in this project [2] contains information about house sales in King County, Washington, including the city of Seattle. Each property is described by a number of features such as living area, number of bedrooms, number of bathrooms, number of floors, property condition, geographical location, and other relevant attributes. The target variable of interest is the selling price of each property.

The primary objective of this project is to develop and evaluate a predictive regression model capable of estimating house prices based on the available property features. Throughout the project, multiple machine learning techniques will be explored and compared in order to identify the most effective approach for the problem.

To achieve this goal, a complete analytical workflow will be followed. The dataset will first be examined and prepared for analysis, after which regression models will be developed and evaluated using appropriate performance metrics. Advanced techniques such as polynomial feature generation, regularization methods, cross-validation, pipelines, and hyperparameter tuning will also be applied to improve model performance and generalization.

The successful completion of this project will result in a predictive model capable of providing accurate house price estimations for unseen properties. Such a model would support data-driven decision making in real estate investments and demonstrate a practical understanding of the end-to-end machine learning process.

2. Dataset Description

The dataset used in this project is the **King County House Sales Dataset** [2], which contains information about residential property sales in King County, Washington, including the city of Seattle. The dataset includes houses sold between May 2014 and May 2015 and is widely used for regression analysis tasks. The primary objective is to predict the selling price of a property based on a collection of physical, geographical, and quality-related characteristics.

The dataset includes the following attributes:

- **ID:** A unique identifier assigned to each property.
- **Date:** The date on which the property was sold.
- **Price:** The selling price of the property expressed in U.S. dollars. This variable serves as the target variable of the prediction task.
- **Bedrooms:** The total number of bedrooms available in the property.
- **Bathrooms:** The total number of bathrooms within the property.
- **Sqft Living:** The interior living area of the house measured in square feet.
- **Sqft Lot:** The total lot area of the property measured in square feet.
- **Floors:** The number of floors or levels in the property.
- **Waterfront:** Indicates whether the property has a waterfront view. Properties without waterfront access are encoded as 0, while waterfront properties are encoded as 1.
- **View:** A numerical rating representing the quality of the property's view.
- **Condition:** A rating that describes the overall condition of the property.
- **Grade:** A quality score assigned according to the King County grading system, reflecting construction quality and design.
- **Sqft Above:** The square footage of the property located above ground level.
- **Sqft Basement:** The square footage of the basement area.
- **Year Built:** The year in which the property was originally constructed.
- **Year Renovated:** The year in which the property was renovated. A value of 0 indicates that no renovation has occurred.
- **Zipcode:** The postal code corresponding to the property's location.
- **Latitude:** The geographical latitude coordinate of the property.
- **Longitude:** The geographical longitude coordinate of the property.
- **Sqft Living15:** The average living area of neighboring properties measured in 2015.
- **Sqft Lot15:** The average lot size of neighboring properties measured in 2015.

The variable **price** serves as the dependent variable that the machine learning models aim to predict. All remaining attributes are treated as independent variables and are used to investigate how property size, quality, location, and other housing characteristics influence residential property values. The dataset provides a realistic real-estate valuation problem and serves as the foundation for all data exploration, model development, and performance evaluation activities conducted throughout this project.

3. Model Implementation

3.1 Environment Setup

```
# Suppress warning messages

def warn(*args, **kwargs):
    pass

import warnings
warnings.warn = warn

# Install required packages (if necessary)

# !pip install -U scikit-learn

import piplite

await piplite.install('seaborn')


# Import required libraries

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

import seaborn as sns


from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler, PolynomialFeatures

from sklearn.linear_model import LinearRegression

%matplotlib inline
```

The implementation of this project was performed within the **IBM Skills Network Labs** [3] environment using Jupyter Notebook. This cloud-based platform provides a preconfigured workspace containing the software packages and machine learning libraries required for the completion of the assignment. Since the notebook relies on the configuration and package versions available in this environment, successful execution outside Skills Network Labs cannot be fully guaranteed without installing the appropriate dependencies and ensuring version compatibility.

The following libraries were imported and utilized throughout the implementation process:

- **Warnings:** Used to suppress non-critical warning messages generated during code execution, resulting in cleaner and more readable notebook outputs.
- **Piplite:** A lightweight package manager available within Skills Network Labs that allows additional Python packages to be installed directly from the notebook environment.
- **Pandas (pd):** A data analysis library used for loading, storing, manipulating, and processing tabular datasets through the DataFrame data structure.
- **NumPy (np):** A numerical computing library that provides support for mathematical operations, multidimensional arrays, and efficient numerical processing.
- **Matplotlib (plt):** A visualization library used to generate plots, charts, and graphical representations of the dataset and model outputs.
- **Seaborn (sns):** A statistical data visualization library built on top of Matplotlib that simplifies the creation of informative and aesthetically appealing visualizations.
- **Pipeline:** A Scikit-learn utility that combines multiple preprocessing and machine learning steps into a single workflow, ensuring a more organized and reproducible implementation.
- **StandardScaler:** A preprocessing technique used to standardize numerical features by transforming them to have zero mean and unit variance.
- **PolynomialFeatures:** A feature engineering tool that generates polynomial and interaction terms from the original input variables, enabling the development of polynomial regression models.
- **LinearRegression:** The implementation of the Ordinary Least Squares (OLS) linear regression algorithm used for predictive modeling and house price estimation.

The combination of these libraries provided the necessary tools for data manipulation, visualization, preprocessing, feature engineering, model construction, and evaluation throughout the development of the housing price prediction system.

3.2 Importing the Dataset

```
from pyodide.http import pyfetch

async def download(url, filename):

    response = await pyfetch(url)

    if response.status == 200:

        with open(filename, "wb") as f:

            f.write(await response.bytes())

filepath=
'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDevelo
perSkillsNetwork-DA0101EN-SkillsNetwork/labs/FinalModule_Coursera/data/kc_hou
se_data_NaN.csv'

await download(filepath, "housing.csv")

file_name = "housing.csv"

df = pd.read_csv(file_name)

df.head()

df.dtypes

df.describe()
```

The second step of the implementation process involved importing the dataset into the Jupyter Notebook environment. Since the notebook was executed within the IBM Skills Network Labs platform, the dataset was not initially stored on the local file system. Therefore, it was necessary to download the dataset from the remote storage location provided by the course.

To achieve this, the `pyfetch` utility was used to retrieve the dataset from the specified URL. A custom asynchronous function named `download()` was created to handle the download process. This function establishes a connection to the remote file

location, retrieves the dataset, and stores it locally as housing.csv within the notebook environment.

After the download process was completed, the dataset was loaded into a Pandas DataFrame using the `read_csv()` function. The resulting DataFrame, named `df`, serves as the primary data structure used throughout the remainder of the project for data exploration, preprocessing, visualization, and model development.

Finally, the `head()` function was executed to display the first five records of the dataset. This initial inspection step allows verification that the dataset has been loaded correctly and provides a preliminary overview of the available attributes and data format.

The successful completion of this stage ensured that the housing dataset was accessible and ready for subsequent analysis and machine learning tasks.

Unnamed: 0		id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
0	0	7129300520	20141013T000000	221900.0	3.0	1.00	1180	5650	1.0	0	...	7	1180	0	1955	0	98178	47.5112	-122.257	1340	5650
1	1	6414100192	20141209T000000	538000.0	3.0	2.25	2570	7242	2.0	0	...	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	2	5631500400	20150225T000000	180000.0	2.0	1.00	770	10000	1.0	0	...	6	770	0	1933	0	98028	47.7379	-122.233	2720	8062
3	3	2487200875	20141209T000000	604000.0	4.0	3.00	1960	5000	1.0	0	...	7	1050	910	1965	0	98136	47.5208	-122.393	1360	5000
4	4	1954400510	20150218T000000	510000.0	3.0	2.00	1680	8080	1.0	0	...	8	1680	0	1987	0	98074	47.6168	-122.045	1800	7503

After successfully loading the dataset, the `dtypes` attribute was used to inspect the data type of each column. This step is particularly important during the early stages of data analysis, as it allows verification that each attribute has been imported using an appropriate data type.

```
Unnamed: 0      int64
id              int64
date           object
price          float64
bedrooms       float64
bathrooms      float64
sqft_living     int64
sqft_lot       int64
floors         float64
waterfront     int64
view           int64
condition      int64
grade          int64
sqft_above     int64
sqft_basement  int64
yr_built       int64
yr_renovated   int64
zipcode        int64
lat            float64
long           float64
sqft_living15  int64
sqft_lot15     int64
dtype: object
```

The output of `dtypes` provides information about whether a variable is stored as an integer (`int64`), floating-point number (`float64`), or object (`object`). Understanding the data types of the available features is essential for identifying potential inconsistencies, detecting incorrectly imported variables, and determining which preprocessing techniques may be required before model development.

For example, numerical attributes such as `price`, `bedrooms`, and `sqft_living` are expected to be represented using numeric data types, while variables containing textual information are typically stored as objects. This inspection step ensures that the dataset structure is consistent with the expected format and suitable for subsequent analysis.

Following the inspection of data types, the `describe()` function was used to generate a statistical summary of the numerical variables contained in the dataset. This function provides a concise overview of the main statistical properties of each feature, allowing a rapid understanding of the dataset without examining every observation individually.

	Unnamed: 0	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...
count	21613.00000	2.161300e+04	2.161300e+04	21600.000000	21603.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000	...
mean	10806.00000	4.580302e+09	5.400881e+05	3.372870	2.115736	2079.899736	1.510697e+04	1.494309	0.007542	0.234303	...
std	6239.28002	2.876566e+09	3.671272e+05	0.926657	0.768996	918.440897	4.142051e+04	0.539989	0.086517	0.766318	...
min	0.00000	1.000102e+06	7.500000e+04	1.000000	0.500000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	...
25%	5403.00000	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	...
50%	10806.00000	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000000	...
75%	16209.00000	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000000	...
max	21612.00000	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000000	...

The generated summary includes the following statistics:

- **Count:** The number of non-missing observations.
- **Mean:** The average value of the variable.
- **Standard Deviation (Std):** A measure of data dispersion around the mean.
- **Minimum (Min):** The smallest observed value.
- **25% Percentile:** The first quartile of the distribution.
- **50% Percentile:** The median value.
- **75% Percentile:** The third quartile of the distribution.
- **Maximum (Max):** The largest observed value.

These statistics provide valuable insights into the distribution of each feature and help identify potential anomalies such as unusually large values, extreme observations, or possible data quality issues. Additionally, the summary allows an initial assessment of the scale and variability of the dataset, which is particularly

useful before performing feature engineering, normalization, or machine learning model training.

By combining the outputs of `dtypes` and `describe()`, a comprehensive preliminary understanding of the dataset structure and numerical characteristics can be obtained before proceeding to more advanced stages of analysis and model development.

3.3 Data Wrangling

```
df.drop(['id', 'Unnamed: 0'], axis=1, inplace=True)

df.describe()

print("number of NaN values for the column bedrooms :",
df['bedrooms'].isnull().sum())

print("number of NaN values for the column bathrooms :",
df['bathrooms'].isnull().sum())

mean=df['bedrooms'].mean()

df['bedrooms'].replace(np.nan,mean, inplace=True)

mean=df['bathrooms'].mean()

df['bathrooms'].replace(np.nan,mean, inplace=True)

print("number of NaN values for the column bedrooms :",
df['bedrooms'].isnull().sum())

print("number of NaN values for the column bathrooms :",
df['bathrooms'].isnull().sum())
```

After importing and inspecting the dataset, a data wrangling process was performed in order to improve data quality and prepare the dataset for machine learning model development. Data wrangling is a crucial stage in the machine learning workflow, as predictive models require clean and consistent data to produce reliable results.

The first step involved removing the columns **id** and **Unnamed: 0** from the dataset. The **id** column represents a unique identifier assigned to each property and does not contain meaningful predictive information. Similarly, the **Unnamed: 0** column corresponds to a previously stored row index and does not describe any characteristic of the properties. Since neither column contributes useful information to the prediction task, both were removed from the dataset.

Following the removal of unnecessary attributes, the `describe()` function was used to generate a statistical summary of the remaining numerical variables. This summary provided information regarding the distribution, central tendency, and variability of each feature, allowing a preliminary assessment of the dataset.

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
count	2.161300e+04	21600.000000	21603.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	5.400881e+05	3.372870	2.115736	2079.899736	1.510697e+04	1.494309	0.007542	0.234303	3.409430	7.656873	1788.390691	291.509045	1971.005136	84.402258	98077.939805	47.560053	-122.213896	1986.552492	12768.455652
std	3.671272e+05	0.926657	0.768996	918.440897	4.142051e+04	0.539989	0.086517	0.766318	0.650743	1.175459	828.090978	442.575043	29.373411	401.679240	53.505026	0.138564	0.140828	685.391304	27304.179631
min	7.500000e+04	1.000000	0.500000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	1.000000	290.000000	0.000000	1900.000000	0.000000	98001.000000	47.155900	-122.519000	399.000000	651.000000
25%	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	3.000000	7.000000	1190.000000	0.000000	1951.000000	0.000000	98033.000000	47.471000	-122.328000	1490.000000	5100.000000
50%	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	1560.000000	0.000000	1975.000000	0.000000	98065.000000	47.571800	-122.230000	1840.000000	7620.000000
75%	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000000	4.000000	8.000000	2210.000000	560.000000	1997.000000	0.000000	98118.000000	47.678000	-122.125000	2360.000000	10083.000000
max	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000000	5.000000	13.000000	9410.000000	4820.000000	2015.000000	2015.000000	98199.000000	47.777600	-121.315000	6210.000000	871200.000000

The next step focused on identifying missing values. Using the `isnull().sum()` method, it was determined that the bedrooms column contained 13 missing values, while the bathrooms column contained 10 missing values. Missing values can negatively affect machine learning algorithms because many models are unable to process incomplete observations directly.

```
number of NaN values for the column bedrooms : 13
number of NaN values for the column bathrooms : 10
```

To address this issue, a mean imputation strategy was adopted. The average value of the bedrooms column was calculated and used to replace all missing entries within that feature. The same procedure was subsequently applied to the bathrooms column. Mean imputation is a commonly used preprocessing technique because it

preserves the number of available observations while introducing minimal distortion to the overall distribution of the data.

Finally, the missing value counts were calculated again to verify that the replacement process had been completed successfully. The resulting output confirmed that both variables contained zero missing values, indicating that all incomplete observations had been handled appropriately.

```
number of NaN values for the column bedrooms : 0  
number of NaN values for the column bathrooms : 0
```

The data wrangling process successfully removed non-informative attributes and eliminated all missing values from the bedrooms and bathrooms features. As a result, the dataset became more suitable for subsequent exploratory analysis and machine learning model training, ensuring that incomplete records would not negatively influence model performance.

3.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to investigate the characteristics of the dataset, identify patterns, examine relationships between variables, and gain a better understanding of the factors that may influence housing prices. This stage is particularly important before model development, as it provides valuable insights regarding the distribution of the data and the relationships among the available features.

Analysis of Floor Distribution

```
df['floors'].value_counts().to_frame()
```

The first exploratory analysis focused on the **floors** attribute, which represents the number of floors available in each property. The `value_counts()` method was used to count the frequency of each unique value appearing in the column, while the `to_frame()` method converted the resulting Series object into a DataFrame for improved readability.

This analysis allows the identification of the most common property configurations within the dataset and provides a better understanding of the distribution of residential buildings according to the number of floors.

	count
floors	
1.0	10680
2.0	8241
1.5	1910
3.0	613
2.5	161
3.5	8

The output revealed that most properties consist of either **one floor** or **two floors**, while houses with 2.5, 3.0, or 3.5 floors are significantly less common. Specifically, one-floor houses represented the largest category within the dataset, followed by two-floor houses.

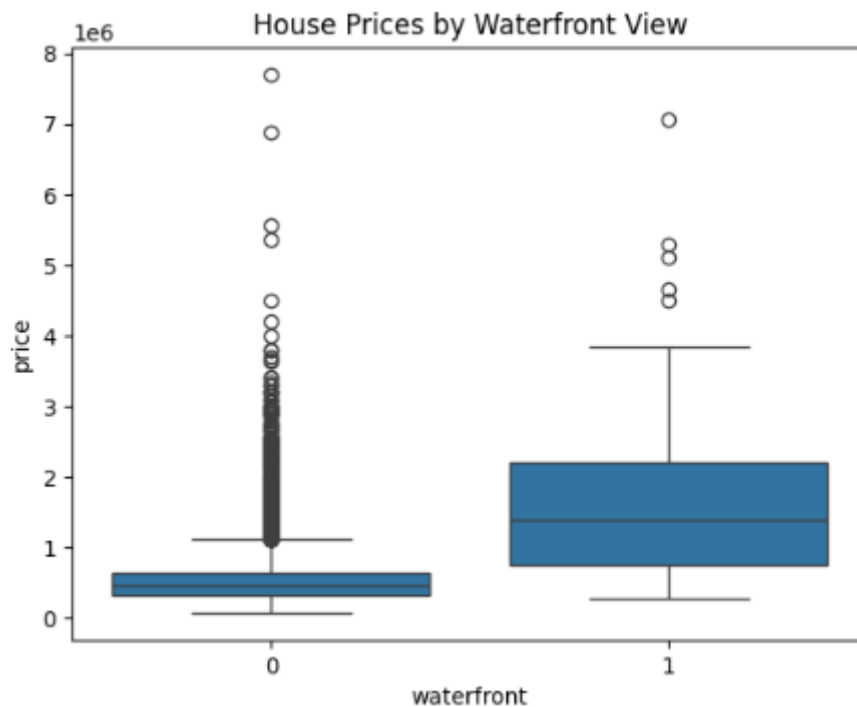
This observation indicates that the majority of residential properties in King County belong to traditional single-story or two-story housing designs.

House Price Distribution by Waterfront View

```
sns.boxplot(x='waterfront',  
            y='price',  
            data=df)  
  
plt.title('House Prices by Waterfront View')  
  
plt.show()
```

A boxplot was generated to compare house prices between properties with and without a waterfront view. The **waterfront** variable is binary, where a value of 0 indicates the absence of a waterfront view and a value of 1 indicates that the property is located near a waterfront.

Boxplots provide useful information regarding the median, interquartile range, data dispersion, and potential outliers within each category. Therefore, they are particularly suitable for comparing distributions between groups.



The resulting visualization demonstrated a substantial difference between the two categories. Properties with a waterfront view generally exhibited significantly higher prices than those without waterfront access.

Furthermore, both categories contained outliers, although waterfront properties tended to display higher extreme values. This observation suggests that location and waterfront access are important factors influencing property value.

The analysis also highlights the presence of several luxury properties with exceptionally high prices, which appear as outliers in the dataset.

Impact of Above-Ground Living Area on House Price

```
sns.regplot(x='sqft_above',
            y='price',
            data=df)

plt.title('Relationship between Sqft Above and Price')

plt.show()
```

To investigate the relationship between above-ground living area and property price, a regression plot was generated using the **sqft_above** feature. This variable represents the square footage of the house located above ground level, excluding basement areas.

The `regplot()` function combines a scatter plot with a fitted regression line, making it possible to visually assess the existence and direction of a potential relationship between the two variables.



The scatter plot revealed a clear positive relationship between **sqft_above** and **price**. As the above-ground living area increases, house prices generally tend to increase as well.

Although some variability is present, the upward trend of the regression line indicates that larger houses are typically associated with higher market values. This finding is consistent with expectations and suggests that living area is an important predictor of housing prices.

Correlation Analysis

```
df_numeric = df.select_dtypes(include=[np.number])  
  
df_numeric.corr()['price'].sort_values()
```

In the final stage of the exploratory analysis, a correlation study was conducted to identify which numerical features exhibit the strongest relationships with the target variable, **price**.

The `select_dtypes()` function was used to isolate numerical variables, while the `corr()` function calculated the Pearson correlation coefficient between all numerical attributes and the target variable. The resulting values were then sorted in ascending order to facilitate interpretation.

```
zipcode      -0.053203  
long         0.021626  
condition    0.036362  
yr_built     0.054012  
sqft_lot15   0.082447  
sqft_lot     0.089661  
yr_renovated 0.126434  
floors       0.256794  
waterfront   0.266369  
lat          0.307003  
bedrooms     0.308797  
sqft_basement 0.323816  
view         0.397293  
bathrooms    0.525738  
sqft_living15 0.585379  
sqft_above   0.605567  
grade        0.667434  
sqft_living  0.702035  
price        1.000000  
Name: price, dtype: float64
```

The correlation analysis revealed that **sqft_living** exhibited the strongest positive correlation with house price, followed by **grade**, **sqft_above**, and **sqft_living15**.

Conversely, variables such as **zipcode** showed very weak correlations with the target variable. The strong correlation observed for **sqft_living** indicates that the overall living area of a property is one of the most influential factors affecting housing prices.

These findings provide valuable guidance for subsequent model development, as highly correlated variables are expected to contribute significantly to predictive performance.

Conclusion of the EDA Phase

The exploratory data analysis phase provided important insights into the structure and characteristics of the housing dataset. The analysis revealed that most properties consist of one or two floors, waterfront properties generally command higher prices, larger above-ground living areas are associated with increased property values, and variables such as **sqft_living**, **grade**, and **sqft_above** demonstrate strong correlations with the target variable. These observations establish a solid foundation for the development and evaluation of predictive regression models in the following stages of the project.

3.5 Model Development

The objective of this phase was to develop and evaluate multiple regression models capable of predicting house prices based on the available property characteristics. Different modeling approaches were investigated, ranging from simple linear regression using a single feature to more complex models incorporating multiple predictors and polynomial transformations. The predictive performance of each model was evaluated using the coefficient of determination (R^2), which measures the proportion of variance in the target variable explained by the model.

Linear Regression Using Living Area

```
x = df[['sqft_living']]  
  
y = df['price']  
  
lm = LinearRegression()  
  
lm.fit(X,y)  
  
lm.score(X,y)
```

A regression model was developed using the **sqft_living** feature, representing the total living area of a property. Since previous exploratory analysis revealed a strong positive correlation between living area and house price, this feature was expected to possess significantly greater predictive power than longitude.

The model achieved:

$$R^2 = 0.49285$$

This result indicates that approximately **49.3%** of the variability in housing prices can be explained solely by the living area of the property.

The substantial improvement compared to the longitude-based model confirms that property size is one of the most influential determinants of housing value.

Multiple Linear Regression

```
features = ["floors", "waterfront", "lat", "bedrooms",  
  
            "sqft_basement", "view", "bathrooms",  
  
            "sqft_living15", "sqft_above", "grade",  
  
            "sqft_living"]  
  
X = df[features]  
  
Y = df['price']  
  
lm = LinearRegression()  
  
lm.fit(X, Y)  
  
lm.score(X, Y)
```

To improve predictive performance, a multiple linear regression model was constructed using eleven explanatory variables associated with property characteristics, location, structural quality, and neighborhood information.

Unlike the previous model, which relied on a single predictor, this approach simultaneously incorporates multiple sources of information that may influence housing prices.

Results

The model achieved:

$$R^2 = 0.65769$$

This result indicates that approximately **65.8%** of the variability in housing prices can be explained by the selected set of features.

Compared to the previous model, the increase in predictive performance demonstrates the importance of combining multiple housing characteristics rather than relying exclusively on living area.

Pipeline-Based Polynomial Regression Model

```
Input = [  
  
    ('scale', StandardScaler()),  
  
    ('polynomial', PolynomialFeatures(include_bias=False)),  
  
    ('model', LinearRegression())  
  
]  
  
pipe = Pipeline(Input)  
  
X = df[features]
```

```
Y = df['price']

pipe.fit(X, Y)

pipe.score(X, Y)
```

In the final stage of model development, a machine learning pipeline was implemented. The pipeline automatically combines data preprocessing, feature engineering, and model training into a single workflow.

The workflow consists of three sequential steps:

1. **StandardScaler** standardizes all numerical features.
2. **PolynomialFeatures** generates polynomial and interaction terms between variables.
3. **LinearRegression** trains a regression model using the transformed feature space.

The introduction of polynomial features allows the model to capture more complex relationships that cannot be represented by a purely linear model.

Results

The pipeline achieved:

$R^2 = 0.75121$

This was the highest performance observed among all evaluated models.

The result indicates that approximately **75.1%** of the variability in housing prices can be explained by the transformed feature set generated by the pipeline.

The improvement over the multiple linear regression model demonstrates that polynomial feature expansion successfully captures additional information and non-linear relationships present within the dataset.

Conclusion of the Model Development Phase

Four different regression approaches were evaluated throughout this phase. The baseline longitude model exhibited almost no predictive capability ($R^2 = 0.00047$), while the living-area model significantly improved performance ($R^2 = 0.49285$). The multiple linear regression model further increased predictive accuracy ($R^2 = 0.65769$) by incorporating multiple property characteristics. Finally, the pipeline-based polynomial regression model achieved the best overall performance ($R^2 = 0.75121$), demonstrating the effectiveness of feature scaling and polynomial feature generation in improving housing price prediction accuracy.

3.6 Model Evaluation and Refinement

The final stage of the project focused on evaluating the predictive performance of the developed models and investigating whether additional techniques could improve generalization on unseen data. To achieve this objective, the dataset was divided into training and testing subsets, allowing model performance to be assessed on observations that were not used during training.

The evaluation process included the application of Ridge Regression as a regularization technique, followed by the combination of Ridge Regression with polynomial feature transformation in order to capture more complex relationships between the available predictors and housing prices.

Train-Test Split

```
features = ["floors", "waterfront", "lat", "bedrooms",  
  
            "sqft_basement", "view", "bathrooms",  
  
            "sqft_living15", "sqft_above", "grade",  
  
            "sqft_living"]  
  
X = df[features]  
  
Y = df['price']  
  
x_train, x_test, y_train, y_test = train_test_split(
```

```
x, y, test_size=0.15, random_state=1)
```

Before evaluating the models, the dataset was partitioned into separate training and testing subsets using the `train_test_split()` function. The training subset was used for model fitting, while the testing subset was reserved exclusively for performance evaluation.

A test size of **15%** was selected, meaning that 85% of the observations were used for training and 15% for testing. Additionally, a fixed random state was specified to ensure reproducibility of the results.

Results

The resulting split produced:

Training Samples: 18,371

Testing Samples: 3,242

This separation allowed the predictive performance of the models to be evaluated on previously unseen observations, providing a more realistic estimate of their generalization capability.

Ridge Regression

```
RidgeModel = Ridge(alpha=0.1)

RidgeModel.fit(x_train, y_train)

RidgeModel.score(x_test, y_test)
```

The first evaluation model utilized **Ridge Regression**, a regularized version of linear regression designed to reduce overfitting. Ridge Regression introduces a penalty term controlled by the regularization parameter α (alpha), which constrains the magnitude of the model coefficients.

For this experiment, the regularization parameter was set to 0.1.

This value applies a mild penalty to the regression coefficients while preserving most of the information contained in the original features.

Results

The model achieved:

$$R^2 = 0.64788$$

This result indicates that approximately **64.8%** of the variance in housing prices can be explained by the selected predictors when evaluated on unseen data.

The performance is slightly lower than the training-based evaluation conducted previously, which is expected since the model is now being assessed on an independent test set. Nevertheless, the model demonstrates satisfactory predictive capability and reasonable generalization performance.

Polynomial Ridge Regression

```
pr = PolynomialFeatures(degree=2)

x_train_pr = pr.fit_transform(x_train)

x_test_pr = pr.transform(x_test)

RidgeModel = Ridge(alpha=0.1)

RidgeModel.fit(x_train_pr, y_train)

RidgeModel.score(x_test_pr, y_test)
```

To further improve predictive performance, a second-order polynomial transformation was applied to both the training and testing datasets. The transformation generated additional polynomial and interaction terms between the original features, allowing the model to capture more complex relationships that cannot be represented by a purely linear model.

After generating the polynomial feature space, a Ridge Regression model with $\alpha = 0.1$ was trained using the transformed training data.

The inclusion of Ridge regularization is particularly important in this scenario because polynomial expansion substantially increases the number of features, which may otherwise lead to overfitting.

Results

The model achieved:

$$R^2 = 0.70027$$

This represents a noticeable improvement over the standard Ridge Regression model.

The increase in predictive performance suggests that the housing dataset contains non-linear relationships and feature interactions that are successfully captured by the polynomial transformation.

Conclusion

The model evaluation phase demonstrated that model performance improves substantially when additional information and more advanced modeling techniques are incorporated. While simple linear regression achieved limited predictive accuracy, the inclusion of multiple explanatory variables significantly enhanced performance.

Furthermore, the application of polynomial feature expansion combined with Ridge regularization produced the strongest generalization performance on unseen data, achieving an R^2 score of **0.70027**. This result indicates that approximately 70% of the variability in housing prices can be explained by the selected housing characteristics and their interactions.

Overall, the findings confirm that both feature engineering and regularization play a crucial role in developing accurate and robust predictive models for real estate price estimation.

4. Conclusion

This project demonstrated the complete development and evaluation of machine learning models for residential house price prediction using real-world real estate data from King County, Washington. The project was conducted as part of the IBM Generative AI Engineering Professional Certificate, with the objective of gaining practical experience in data analysis, feature engineering, regression modeling, and model evaluation techniques.

Throughout the project, a structured machine learning workflow was followed, beginning with data acquisition and preprocessing, followed by exploratory data analysis, feature correlation investigation, model development, and performance evaluation. Particular attention was given to identifying the most influential factors affecting house prices and assessing their predictive capabilities through various regression approaches.

Several predictive models were developed and compared, including simple linear regression, multiple linear regression, Ridge Regression, and polynomial regression models. The experimental results demonstrated that the use of multiple explanatory variables significantly improved predictive performance compared to single-feature models. Furthermore, the incorporation of polynomial feature transformations enabled the models to capture more complex relationships within the dataset, leading to substantial improvements in predictive accuracy.

The evaluation phase highlighted the importance of regularization techniques such as Ridge Regression for controlling model complexity and reducing the risk of overfitting. Additionally, the application of polynomial transformations revealed the presence of non-linear relationships among housing characteristics that could not be adequately represented by purely linear models.

The best-performing model achieved an R^2 score of approximately 0.70 on previously unseen test data, indicating that a large proportion of the variability in house prices can be explained by the selected property attributes and their interactions. This result demonstrates the effectiveness of combining feature engineering and regularization techniques in real-world predictive analytics tasks.

Overall, the project provided valuable hands-on experience in the complete machine learning lifecycle, from data preparation and exploratory analysis to model construction, evaluation, and refinement. The developed models illustrate a practical application of regression-based machine learning techniques in the real estate domain and highlight the potential of data-driven approaches for supporting property valuation and investment decision-making processes.

References

1. <https://www.coursera.org/professional-certificates/ibm-generative-ai-engineering>, “IBM Generative AI Engineering”, Coursera.
2. <https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>, “House Sales in King County, USA”.
3. <https://skills.network/lab-tools>, “Skills Network Labs”, IBM.